

ON-PREMISES MLOPS COCKPIT

ON-PREMISES MLOPS COCKPIT



TECHNICAL ARCHITECTURE

Table of Contents

1. Overview.....	3
2. Repository layout	4
3. Backend architecture	5
3.1 Routers	5
3.2 Managers	5
3.3 MCP layer	6
3.4 Request lifecycle	7
4. Frontend architecture.....	8
5. Service integrations	9
5.1 DVC	9
5.2 Hydra	9
5.3 MLflow	10
5.4 Airflow.....	11
5.5 MinIO	12
5.6 Evidently	13
5.7 The Assistant	13
5.8 Model serving (noted-serving)	15
6. Deployment	16
7. Cross-cutting concerns	17
7.1 State scope	17
7.2 Streaming.....	17
7.3 Testing	17
8. Extending noted	18

1. Overview

noted is a browser-hosted ML engineering workbench: a notebook editor on top of a polyglot runtime, wired to an MLOps service mesh (experiment tracking, artefact versioning, pipeline orchestration, drift monitoring, object storage) and fronted by an AI Assistant that can drive the platform via native tool calls.

The backend is a FastAPI application that orchestrates Jupyter kernels, delegates to external services over their own APIs, and streams state to a single-page vanilla JS frontend over Socket.IO. The Assistant uses either Anthropic Claude (hosted) or a local Gemma 4 (via a companion agent_server), selected per-request.

Layer	Stack
Backend	Python 3.12, FastAPI, Socket.IO (Python async), Uvicorn, Jupyter kernels
Frontend	Vanilla ES modules, CodeMirror 6, xterm.js, Wunderbaum, Monaco editor (for file tabs)
Services	MLflow, Airflow, MinIO, Evidently, knowledge graph, Postgres, Redis, TF Serving clone
Assistant	Anthropic API + agent_server (llama-cpp-python running Gemma 4)
Runtime	Docker Compose (one compose project per environment: cpu, gpu, local)

2. Repository layout

```
noted/
  backend/          FastAPI application
    app/
      main.py        App bootstrap + Socket.IO events + lifespan
      config.py       DATA_DIR / PROJECTS_DIR / MOUNTS_DIR resolution
      routers/        HTTP endpoints, one file per feature area
      managers/       Domain logic and external-service clients
      mcp/            MCP tool schemas, Gemma native tool parser, context
  router
  frontend/         Static assets served by FastAPI
    index.html
    js/             ~90 ES modules (panels, editors, clients, services)
    css/
  services/         Docker Compose files (cpu, gpu, local)
  client/           Separate image: TF Serving wrapper (noted-serving)
  data/            Bind-mounted runtime state
    projects/        User projects
    mounts/          External git repos mounted read/write
    skills/          Assistant skill library (SKILL.md files)
    templates/       Notebook / file templates
    chat_history/    Per-project Assistant memory
    testing/         Test harness reports + scenario definitions
  testing/assistant/
    architecture/    Harness design docs
    harness/         Python harness (driver, reporter, deterministic, judge)
    skills/ tools/   YAML+MD scenario specs (per-skill / per-tool)
  documents/        Design documents (this file lives in documents/architecture/)
```

Two rules about this tree:

- backend/ and frontend/ are COPY'd into the image. Changes require docker compose up -d --build.
- data/ is bind-mounted. Edits to skills, templates, and per-project files take effect without rebuild (registries that cache in memory still need a restart; the SkillRegistry is the main one).

3. Backend architecture

FastAPI is the HTTP frontend. The application is wired in three layers.

3.1 Routers

Under `backend/app/routers/`, one file per feature area. Each router declares a prefix and a set of endpoints, pulls in a manager singleton, and keeps HTTP-specific concerns (request/response models, streaming, status codes) out of the business layer.

Router	Prefix	Responsibility
<code>notebooks.py</code>	<code>/api/notebooks</code>	CRUD, cell ops, kernel lifecycle proxy
<code>files.py</code> / <code>documents.py</code>	<code>/api/files</code> , <code>/api/documents</code>	Filesystem read/write, markdown/PDF export
<code>projects.py</code>	<code>/api/projects</code>	Project discovery, git init, metadata
<code>dvc.py</code>	<code>/api/dvc</code>	Track, push/pull, status, lineage
<code>hydra.py</code>	<code>/api/hydra</code>	Schema, compose, bundle load (incl. MLflow source)
<code>mlflow.py</code>	<code>/api/mlflow</code>	Experiments, runs, artefacts, metrics
<code>airflow.py</code>	<code>/api/airflow</code>	DAG discovery, trigger, poll, logs
<code>evidently.py</code>	<code>/api/evidently</code>	Project / report proxy, drift + quality health
<code>minio.py</code>	<code>/api/minio</code>	Bucket / object browsing
<code>serving.py</code> / <code>registry.py</code>	<code>/api/serving</code> , <code>/api/registry</code>	Deploy/stop, invoke, register model
<code>llm.py</code>	<code>/api/llm</code>	Assistant chat + write-tool confirmation
<code>lsp.py</code> / <code>dap.py</code>	<code>/api/lsp</code> , <code>/api/dap</code>	Proxy to language servers and debug adapters
<code>graph_proxy.py</code>	<code>/api/graph</code>	Pass-through to the knowledge graph service

All long-running or stateful work streams over Socket.IO instead (see `main.py` events: `kernel:execute`, `lsp:*`, `dap:*`, `pipeline:task_status`, terminal PTYs, etc.).

3.2 Managers

Under `backend/app/managers/`. A manager owns one subsystem: it holds a client, a cache, or a subprocess pool, exposes typed methods, and hides vendor SDKs behind stable signatures. Routers instantiate managers as module-level singletons.

Pattern is the same across the three integration styles:

- **REST-client managers** (`AirflowManager`, `EvidentlyManager`, `MinioManager`) - a `requests/httpx/botocore` session plus thin method wrappers.
- **SDK-wrapper managers** (`MLflowManager`) - lazy `MLflowClient`, warmed on startup, uniform dict serialization.

- **Subprocess managers** (DvcManager, GitManager, VenvManager) - subprocess.run with validated cwd and credential injection.

The key managers beyond services are:

- `KernelManagerService` + `ExecutionBridge` - Jupyter kernel lifecycle and execute protocol, bridging ZMQ to Socket.IO.
- `AutoInstrumentation` - generates Python snippets injected into the kernel to monkey-patch MLflow for live metrics and run bookkeeping.
- `HydraCache` - in-memory (`notebook_uid`, `run_id`) -> bundle bytes cache so Time Machine loads don't re-fetch from MLflow.
- `ProjectRegistry` - single source of truth that resolves a `project_id` to a filesystem path; recognizes both local projects and mounted repos.
- `LLMRouter` / `LLMContext` / `LLMSkills` / `LLMTools` - split responsibilities of the Assistant (see section 5.7).

3.3 MCP layer

backend/app/mcp/ is the Assistant's tool surface.

- `tools.py` declares 34 tools (25 read, 9 write) with JSON schemas using the mcp types.
- `mount.py` exposes them over Streamable HTTP at `/mcp` so external MCP clients can consume them.
- `gemma_tool_parser.py` parses Gemma 4's native `<|tool_call>call:name{key:<|">value<|">,...}<|tool_call>` format back into `{name, args}` dicts. This is not a regex - it's a delimiter-aware scanner because `}` legitimately appears inside string values (Python code in content args).

```
# gemma tool parser.py - find matching brace, the core of the scanner
```

```
def _find_matching_brace(text: str, start: int) -> int:
```

```
i = start; n = len(text); dlen = len( DELIMITER)
```

```
while i < n:
```

```
if text[i:i + dlen] == DELIMITER:
```

```
i += dlen
```

```
while i < n:
```

```
if text[i:i + dlen] == DELIMITER:
```

```
i += dlen
```

break

```
i += 1
```

```
elif text[i] == '}':
```

```
return i
```

```
else:
```

```
        i += 1
    return -1
```

- context_router.py filters the tool list per request based on workspace context (for Anthropic: saves ~2000 tokens/turn; local LLM gets the full set).

3.4 Request lifecycle

Chat request through /api/llm/chat:

1. Router parses ChatRequest (message, context_descriptor, temperature, max_tokens).
2. LLMContext.build_context_message() reads workspace state from managers (notebook cells, MLflow runs, Hydra config, files), emits “active conditions” used to gate priority-1 skills.
3. LLMSkills.get_static_skills(conditions) returns matched skill bodies, enforced under a 16000-token budget.
4. LLMRouter selects the active model (Anthropic or agent_server), streams tokens via chat_stream.
5. Tokens are parsed on the fly into full_text; Gemma native tool calls are captured by parse_gemma_tool_calls.
6. Read tools execute inline (up to MAX_TOOL_ROUNDS = 6, feeding results back into messages).
7. Write tools stop the stream with pending_action; client calls /api/llm/confirm to approve, which executes and resumes.
8. SSE events along the way: token, skills, tool_badge, tool_result, pending_action, context_block, usage, [DONE].

4. Frontend architecture

Plain ES modules under frontend/js/. No build step, no framework. Modules are loaded directly by index.html. The app has four canonical surfaces:

Surface	Entry	Key modules
Notebook editor	app-notebooks.js	CellEditor, CellOutput, TocPanel, KernelClient
File editor	app-file-editors.js	FileEditor, Monaco wrapper, DiffView
Chat	app-chat.js	ChatPanel, ChatService, InfoBar
Explorer (left tree)	app.js	Wunderbaum tree, MLflow / Hydra / DVC / Airflow / Evidently views

Service clients (KernelClient, AgentClient, DebugClient) wrap fetch() and Socket.IO events with typed promises + event subscribers. The InteractiveTerminal attaches xterm.js to a server-side PTY via Socket.IO.

State is held in plain module-scope variables with explicit update paths; a DestroyRegistry pattern ensures panels register their destroy() functions so the outer frame can tear down sub-surfaces without leaks (see feedback_event_listener_cleanup.md).

5. Service integrations

This section is the focus of the document: for each integration, where the code lives, the client pattern, and the data model.

5.1 DVC

- File: [backend/app/managers/dvc_manager.py](#) (542 lines)
- Router: [backend/app/routers/dvc.py](#)
- Remote: MinIO bucket noted-dvc (same process, different container).
- Pattern: subprocess wrapper. DVC itself is a CLI tool; noted shells out rather than re-implementing it.

```
# dvc_manager.py
MINIO_ENDPOINT = os.environ.get('DVC_MINIO_ENDPOINT', 'http://noted-minio:9000')
MINIO_ACCESS_KEY = os.environ.get('DVC_MINIO_ACCESS_KEY', 'admin')
MINIO_SECRET_KEY = os.environ.get('DVC_MINIO_SECRET_KEY', 'password')
MINIO_BUCKET = os.environ.get('DVC_MINIO_BUCKET', 'noted-dvc')

def _run(self, args, cwd, check=False, timeout=None):
    env = os.environ.copy()
    env['AWS_ACCESS_KEY_ID'] = MINIO_ACCESS_KEY
    env['AWS_SECRET_ACCESS_KEY'] = MINIO_SECRET_KEY
    return subprocess.run(
        args, cwd=cwd, capture_output=True, text=True, check=check,
        env=env, timeout=timeout,
    )
```

Tracked extensions are whitelisted (DVC_EXTENSIONS) so only real data/model/media files are candidates for dvc add. The manager caches status per repo for 5s to survive UI refresh storms.

Lineage: when the kernel executes a training cell, AutoInstrumentation tags the MLflow run with dataset.hash = <md5> for each tracked file the code reads.

The Explorer's DVC panel renders a run-to-file graph by joining MLflow tags with .dvc files on disk.

5.2 Hydra

File: [backend/app/managers/hydra_manager.py](#), [hydra_source.py](#), [hydra_cache.py](#)
Router: [backend/app/routers/hydra.py](#)

Hydra is not running as a service - the manager re-implements the composition rules that Hydra would apply at CLI-invocation time, so noted can:

- Show the user the schema (group options + leaf overrides).
- Compose a resolved config before a run.
- Archive the exact config bundle alongside the MLflow run for reproducibility.
- Restore an archived bundle and re-run the same experiment (“Time Machine”).

The abstraction is HydraSource:

```
# hydra_source.py - two concrete sources, same interface
class HydraSource:
    def exists(self) -> bool: ...
    def read(self, rel_path: str) -> bytes: ...
    def walk(self) -> Iterable[tuple[str, list[str], list[str]]]: ...

class LocalSource(HydraSource):    # reads from data/projects/<id>/config/
    ...

class MlflowSource(HydraSource):    # reads from an archived run's hydra/ artefacts
    ...
```

assemble_bundle_from_source(source, group_selections, overrides) is the one place that actually builds a bundle; both the per-run auto-archival path and the Composer’s snapshot path use it. This avoids drift between “what we archive” and “what we restore”.

HydraCache keys on (notebook_uid, run_id) and is scoped to one running process. Warm cache hits let Time Machine swap baselines without hitting MLflow on every interaction.

5.3 MLflow

File: `backend/app/managers/mlflow_manager.py` (686 lines)

Instrumentation: `backend/app/managers/auto_instrumentation.py`

Router: `backend/app/routers/mlflow.py`, `registry.py`, `serving.py`

Pattern: thin SDK wrapper. MlflowClient is lazily created, warmed on startup, and reused.

```
# mlflow_manager.py
MLFLOW_TRACKING_URI = os.environ.get("MLFLOW_TRACKING_URI", "http://mlflow:5000")

def _get_client(self):
    if self._client is None:
        from mlflow.tracking import MlflowClient
        self._client = MlflowClient(tracking_uri=MLFLOW_TRACKING_URI)
    return self._client
```

The interesting part is the **kernel-side instrumentation** generated by AutoInstrumentation. Before a notebook cell runs, noted injects a silent Python snippet that monkey-patches `mlflow.log_metric` and `mlflow.log_metrics` to additionally emit an IPython `display_data`

payload with MIME type `application/x-noted-metric`. The notebook execution bridge intercepts that MIME and pushes it to the frontend as a live chart update.

```
# auto_instrumentation.py - silent injected code (excerpt)
def __noted_log_metric(key, value, step=None, **kw):
    __orig_log_metric(key, value, step=step, **kw)
    try:
        run = __mlf_hook.active_run()
        rid = run.info.run_id if run else None
        __ipy_display({"application/x-noted-metric": __json_hook.dumps({
            "run_id": rid, "key": key, "value": float(value),
            "step": step, "timestamp": __time_hook.time()
        })}, raw=True)
    except Exception:
        pass
```

Run Manager additionally injects a scaffolding block (`RUN_START_CODE`) that opens an MLflow run bound to a named experiment, tags it (`instrumentation=experiments`, `dataset.hash=...`, `noted.hydra_config_hash=...`), and closes it at the end. That wraps unmodified user code in deterministic start/end semantics without requiring the user to write `mlflow.start_run()`.

Model registration + serving: the registry router calls `client.create_registered_model / create_model_version` against the tracking server. serving routes `/api/serving/deploy` to the companion noted-serving container (see section 5.8).

5.4 Airflow

File: `backend/app/managers/airflow_manager.py` (466 lines)
Router: `backend/app/routers/airflow.py`

Pattern: REST client against Airflow API v2. JWT token obtained on demand and refreshed on 401.

```
# airflow_manager.py
def _ensure_token(self):
    if self._token and time.time() < self._token_expiry - 60:
        return
    session = self._get_session()
    for path in [f'{AIRFLOW_BASE_PATH}/auth/token', '/auth/token']:
        resp = session.post(
            f'{AIRFLOW_API_URL}{path}',
            json={'username': AIRFLOW_USERNAME, 'password': AIRFLOW_PASSWORD},
        )
        if resp.status_code in (200, 201):
            data = resp.json()
```

```
self._token = data.get('access_token')
self._token_expiry = time.time() + data.get('expires_in', 1800)
session.headers['Authorization'] = f'Bearer {self._token}'
return

# Fallback: basic auth
session.auth = (AIRFLOW_USERNAME, AIRFLOW_PASSWORD)
```

DAGs live in each project's dags/ folder. Airflow is configured to scan /opt/airflow/dags, which is a bind mount over the union of all project dags/ directories.

A new .py file in a project shows up in Airflow within one scan interval (30 seconds by default).

Trigger + monitor: the /api/airflow/dags/{dag_id}/trigger endpoint POSTs to /dags/{dag_id}/dagRuns with user-supplied params (Composer-composed config JSON), then spawns _poll_run_status as an asyncio task.

That task polls get_run_state + get_task_instances every 4s for up to 10 minutes and emits pipeline:task_status Socket.IO events so the UI can animate the DAG graph.

Training DAGs follow the canonical compose -> ingest -> preprocess -> train -> promote pattern (with optional log_hydra_lineage fan-out to preserve the config bundle next to the run).

5.5 MinIO

File: [backend/app/managers/minio_manager.py](#) (132 lines)
Router: [backend/app/routers/minio.py](#)

Pattern: botocore S3 client. No boto3 dependency - botocore is already present via DVC's S3 extra.

```
# minio_manager.py
def _s3(self):
    if self._client is None:
        import botocore.session
        session = botocore.session.get_session()
        self._client = session.create_client(
            's3',
            endpoint_url=MINIO_ENDPOINT,
            aws_access_key_id=MINIO_ACCESS_KEY,
            aws_secret_access_key=MINIO_SECRET_KEY,
        )
    return self._client
```

MinIO fills three roles in the stack:

Bucket	Producer	Consumer
noted-dvc	dvc push (subprocess)	dvc pull on other machines
mlflow	MLflow server (artefacts)	MLflow client (via tracking server)
user buckets	Manually via Explorer	Anything

MinIO is not on the Assistant's tool surface; users browse it through the Explorer tree. Because MinIO exposes the S3 API, anything that can speak S3 (polars, duckdb, aws-cli) can read/write directly.

5.6 Evidently

File: [backend/app/managers/evidently_manager.py](#) (150 lines)

Router: [backend/app/routers/evidently.py](#)

Pattern: async HTTP client (`httpx.AsyncClient`) against the Evidently workspace API. No dependency on the evidently Python package in the backend; the heavy lifting happens inside the Evidently service container.

```
# evidently_manager.py
EVIDENTLY_URL = "http://noted-evidently:8000"

async def list_projects(self) -> list[dict]:
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        resp = await client.get(f"{self._base_url}/api/projects")
        resp.raise_for_status()
        return resp.json()
```

Reports are generated from inside user code (notebook cells or DAG tasks) using the Evidently SDK, then pushed to the workspace API under a project. Noted reads them back for the Explorer's health-status badges and for drift alerts in the chat context.

`get_drift_status(project_id)` buckets `dataset_drift_share` into green/yellow/red thresholds (0.2 / 0.5) to drive the drift indicator dot next to each project in the tree. The actual report HTML is displayed in a panel via an `<iframe>` pointing at the Evidently UI.

5.7 The Assistant

Files:

- Router: [backend/app/routers/llm.py](#) (1078 lines)
- Routing: [backend/app/managers/llm_router.py](#)
- Context: [backend/app/managers/llm_context.py](#)
- Skills: [backend/app/managers/llm_skills.py](#)
- Tools: [backend/app/managers/llm_tools.py](#)

- Memory: [backend/app/managers/llm_memory.py](#)
- Gemma parser: [backend/app/mcp/gemma_tool_parser.py](#)
- Tool schemas: [backend/app/mcp/tools.py](#)

Split-of-concerns:

Module	Responsibility
llm_router	Picks Anthropic vs local agent_server; uniform chat_stream iterator
llm_context	Builds the WORKSPACE CONTEXT block; emits active-condition set
llm_skills	Loads skill library from data/skills/; gates priority-1 skills by conditions
llm_tools	Prepares write actions, executes read tools, dispatches approved writes
llm_memory	Per-project conversation memory, compaction when over budget

Skills are auto injected at priority 1 when their triggers list matches the current conditions:

```
# llm_skills.py
def get_static_skills(self, context_conditions):
    matched = []
    total_tokens_est = 0
    max_budget = 16000
    for skill in self._skills.values():
        if skill.priority != 1 or not skill.triggers:
            continue
        if any(t in context_conditions for t in skill.triggers):
            est_tokens = len(skill.content.split()) * 1.3
            matched.append((skill.name, skill.content))
            total_tokens_est += est_tokens
    if total_tokens_est > max_budget:
        raise RuntimeError(
            f"Auto-injected priority-1 skills exceed {max_budget}-token budget..."
        )
    return matched
```

Conditions come from llm_context._get_matched_skills, built from the request's context descriptor and the already-assembled context blocks. Current emitters:

- workspace_active, notebook_cell_selected, file_open_in_editor
- mlflow_experiment_in_context, mlflow_run_in_context, mlflow_run_failed
- hydra_config_in_context, hydra_and_dag_in_context
- linter_rule

- Filesystem-derived signals: `airflow_in_context`, `dvc_in_context`, `evidently_in_context`, `hydra_config_in_context` (gate priority-1 skills for that surface without bloating non-relevant chats)

Write tools (`update_cell`, `insert_cell`, `batch_update_cells`, `update_file`, `create_file`, `fix_lint_issues`) never execute inline - they always produce a `pending_action` SSE frame that the frontend turns into a confirmation panel. The follow-up stream from `/api/llm/confirm` is merged with the initial stream for a single logical turn.

The local model path lives in a companion project `agent_server` (llama-cpp-python serving Gemma 4 with a custom chat template). The `LLMRouter` targets it via HTTP when the user selects a local model. Same tool schemas, same SSE contract - the Gemma native tool-call syntax is translated by `gemma_tool_parser.py` so the rest of the system sees identical `{name, args}` dicts regardless of provider.

5.8 Model serving (noted-serving)

Companion image: `client/` directory, deployed as `noted-serving` container.
Router: [backend/app/routers/serving.py](#), [client/app/predict.py](#)

The serving container is a minimal FastAPI app that loads registered MLflow models and exposes `/predict`. Deployment is triggered from the Explorer (or via the Assistant's `deploy_model` tool); the backend POSTs a load request to the serving container, which:

1. Resolves the requested (`model_name`, `version_or_alias`) to an MLflow logged-model URI.
2. Calls `mlflow.pyfunc.load_model(...)`.
3. Swaps the loaded model atomically so in-flight requests finish on the old one.

Inference is a `pyfunc predict()` call. For models with signature-enforced inputs (TensorSpec named tensors), the serving wrapper unwraps `{"input": ndarray}` payloads before the model sees them, so the serving layer works uniformly across `keras/pyfunc/raw-python` models.

6. Deployment

Three Compose files under services/:

File	Use
docker-compose.yml	Base services (noted, mlflow, minio, airflow, evidently, postgres, redis, graph, serving)
docker-compose.gpu.yml	Adds GPU device reservations to noted + serving
docker-compose.local.yml	Developer-only port exposures, extra bind mounts

Plus a generated data/docker-compose.mounts.yml that declares the list of external git-repo bind mounts (one volume mapping per project).

Typical dev workflow:

```
# Edit backend/ or frontend/, then:
docker compose \
  -f services/docker-compose.yml \
  -f services/docker-compose.gpu.yml \
  -f data/docker-compose.mounts.yml \
  up -d --build noted
```

Only noted is usually rebuilt; the other services are vendor images. data/ bind-mounted content (skills, templates, per-project files, user projects) is live across rebuilds. The SkillRegistry is a process-level singleton, so edits to data/skills/*/SKILL.md require a restart of the noted container to be picked up.

7. Cross-cutting concerns

7.1 State scope

Scope	Storage
Request-local	FastAPI dependency / function locals
Connection-local	Socket.IO sid -> client_rooms[sid][room_key]
Per-project	data/projects/<id>/noted/ + data/chat_history/<id>/
Per-container (in-memory)	Singleton managers, registries, caches
Global, persistent	Bind-mounted data/, service volumes (mlflow, minio, postgres)

7.2 Streaming

Everything that takes longer than ~1s streams: LLM chat (SSE), kernel execute (Socket.IO), Airflow run polling (Socket.IO), pipeline task updates (Socket.IO), log tails (Socket.IO). Polling from the frontend is forbidden; backend pushes. The SSE contract for chat is documented by the tagged payload types (token, tool_badge, tool_result, pending_action, skills, context_block, usage, error, [DONE]).

7.3 Testing

Harness lives at testing/assistant/. Two-layer evaluation:

- Layer 1 (deterministic): did the Assistant call the expected tools with the expected arg shapes?
- Layer 2 (LLM-as-Judge): does the natural-language answer meet the scenario's focus? Judge is a local Gemma model with a structured rubric (tool_call_check, answer_check, procedural_check).

Scenarios are per-skill and per-tool YAMLs that pair with human-readable MD docs. Results accumulate in data/testing/reports/_history.csv (one latest-state row per scenario). A static HTML dashboard (_progress.html) polls the CSV + scenario manifest + a _current.json marker and renders progress stats.

8. Extending noted

Adding a new integration typically means:

1. **Compose:** declare a service in `services/docker-compose.yml` on `noted-network`.
2. **Manager:** `backend/app/managers/<name>_manager.py` with a single class, lazy client init, and uniform dict returns.
3. **Router:** `backend/app/routers/<name>.py` exposing `/api/<name>/*`.
4. **Frontend:** a panel/client pair under `frontend/js/`; subscribe to any Socket.IO events from the router.
5. **Context signal** (optional): emit a condition from `llm_context._get_matched_skills` when the feature is present so Assistant skills about that surface can auto-inject.
6. **Skill** (optional): drop a `SKILL.md` under `data/skills/<name>/` with a `triggers:` list matching your condition and `priority: 1` to have it auto-injected.
7. **Tool** (optional): add read/write tool schemas to `backend/app/mcp/tools.py` and dispatch in `backend/app/managers/llm_tools.py`.

Each of the seven service integrations above follows this shape. New ones should too.